

CSE 302

Lecture 2, Software Engineering

- ▶ Dr. Ashraf Mahroos, Ph.D
- ▶ Professor of Engineering Systems
- ▶ The figures and text included in slides are borrowed from various books, websites, and other sources for academic purposes only. The author do not claim any originality.

Software engineering fundamentals



- ✧ Some **fundamental principles** apply to all types of software system, irrespective of the development techniques used:
 - Systems should be developed using a **managed and understood development process**. Of course, different processes are used for different types of software.
 - **Dependability and performance** are important for all types of system
 - Understanding and managing the **software specification and requirements** (what the software should do) are important
 - Where appropriate, you should **reuse software** that has already been developed rather than write new software

Software engineering and the web



- ✧ **The Web** is now a platform for running application and organizations are increasingly developing web-based systems rather than local systems
- ✧ Web services (discussed in Chapter 19) allow application functionality to be accessed over the web
- ✧ Cloud computing is an approach to the provision of computer services where applications run remotely on the 'cloud'
 - Users do not buy software but pay according to use

Web software engineering



- ✧ **Software reuse** is the dominant approach for constructing web-based systems
 - When building these systems, you think about how you can assemble them from pre-existing software components and systems
- ✧ **Web-based systems** should be developed and delivered incrementally
 - It is now generally recognized that it is impractical to specify all the requirements for such systems in advance
- ✧ **User interfaces**, constrained by capabilities of web browsers
 - Technologies such as AJAX allow rich interfaces to be created within a web browser but are still difficult to use. Web forms with local scripting are more commonly used.

Asynchronous JavaScript and XML (**Ajax**) refer to a group of technologies that are used to **develop web** applications. By combining these technologies

Web-based software engineering



- ✧ Web-based systems are complex distributed systems but the fundamental principles of software engineering discussed previously are as applicable to them as they are to any other types of system
- ✧ The fundamental ideas of software engineering, discussed in the previous section, apply to web-based software in the same way that they apply to other types of software system

NEED OF SOFTWARE ENGINEERING

The need of software engineering arises because of higher rate of change in user requirements and environment on which the software is working.

- **Large software** - It is easier to build a wall than to a house or building, likewise, as the size of software become large engineering has to step to give it a scientific process.
- **Scalability**- If the software process were not based on scientific and engineering concepts, it would be easier to re-create new software than to scale an existing one.
- **Cost**- As hardware industry has shown its skills and huge manufacturing has lower down the price of computer and electronic hardware. But the cost of software remains high if proper process is not adapted.
- **Dynamic Nature**- The always growing and adapting nature of software hugely depends upon the environment in which the user works. If the nature of software is always changing, new enhancements need to be done in the existing one. This is where software engineering plays a good role.
- **Quality Management**- Better process of software development provides better and quality software product.

CHARACTERISTICS OF GOOD SOFTWARE

A software product can be judged by what it offers and how well it can be used. This software must satisfy on the following grounds:

- Operational
- Transitional
- Maintenance

Well-engineered and crafted software is expected to have the following characteristics:

Operational

This tells us how well software works in operations. It can be measured on:

- Budget
- Usability
- Efficiency
- Correctness
- Functionality
- Dependability
- Security
- Safety

Transitional

This aspect is important when the software is moved from one platform to another:

- Portability
- Interoperability
- Reusability
- Adaptability

Maintenance

This aspect briefs about how well a software has the capabilities to maintain itself in the ever-changing environment:

- Modularity
- Maintainability
- Flexibility
- Scalability

In short, Software engineering is a branch of computer science, which uses well-defined engineering concepts required to produce efficient, durable, scalable, in-budget and on-time software products



Key points [Professional Software Development]

- ✧ **Software engineering** is an engineering discipline that is concerned with all aspects of software production
- ✧ Essential **software product attributes** are maintainability, dependability and security, efficiency and acceptability
- ✧ The **high-level activities** of specification, development, validation and evolution are part of all software processes
- ✧ The **fundamental notions** of software engineering are **universally applicable** to all types of system development

Key points [Professional Software Development]



- ✧ There are **many different types of system** and each requires appropriate software engineering tools and techniques for their development
- ✧ The **fundamental ideas** of software engineering are **applicable** to all types of software system

Software engineering ethics



- ✧ Software engineering involves wider responsibilities than simply the application of technical skills
- ✧ Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals
- ✧ Ethical behaviour is more than simply upholding the law but involves following a set of principles that are morally correct

Issues of professional responsibility

✧ Confidentiality

- Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed

✧ Competence

- Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence



Issues of professional responsibility

✧ Intellectual property rights

- Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.

✧ Computer misuse

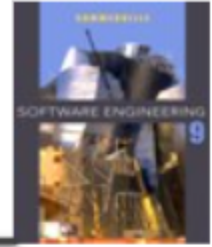
- Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

ACM/IEEE Code of Ethics



- ✧ The professional societies in the US have cooperated to produce a code of ethical practice
- ✧ Members of these organisations sign up to the code of practice when they join
- ✧ **The Code** contains **eight Principles** related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of the profession

The ACM/IEEE Code of Ethics



Software Engineering Code of Ethics and Professional Practice

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

PREAMBLE

The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

Ethical principles



1. PUBLIC - Software engineers shall act consistently with the public interest.
2. CLIENT AND EMPLOYER - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. PRODUCT - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. JUDGMENT - Software engineers shall maintain integrity and independence in their professional judgment.
5. MANAGEMENT - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. PROFESSION - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. COLLEAGUES - Software engineers shall be fair to and supportive of their colleagues.
8. SELF - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Ethical dilemmas



- ✧ Disagreement in principle with the policies of senior management
- ✧ Your employer acts in an unethical way and releases a safety-critical system without finishing the testing of the system
- ✧ Participation in the development of military weapons systems or nuclear systems

Case studies



✧ A personal insulin pump

- An embedded system in an insulin pump used by diabetics to maintain blood glucose control

✧ A mental health case patient management system

- A system used to maintain records of people receiving care for mental health problems

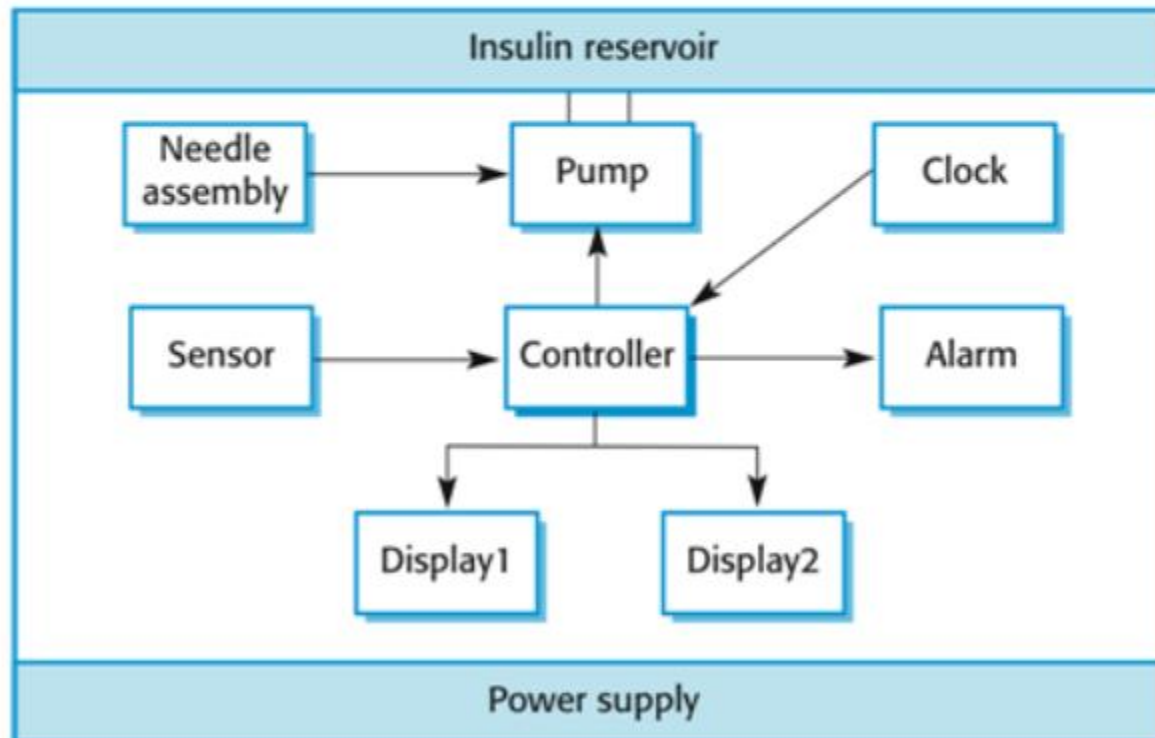
✧ A wilderness weather station

- A data collection system that collects data about weather conditions in remote areas

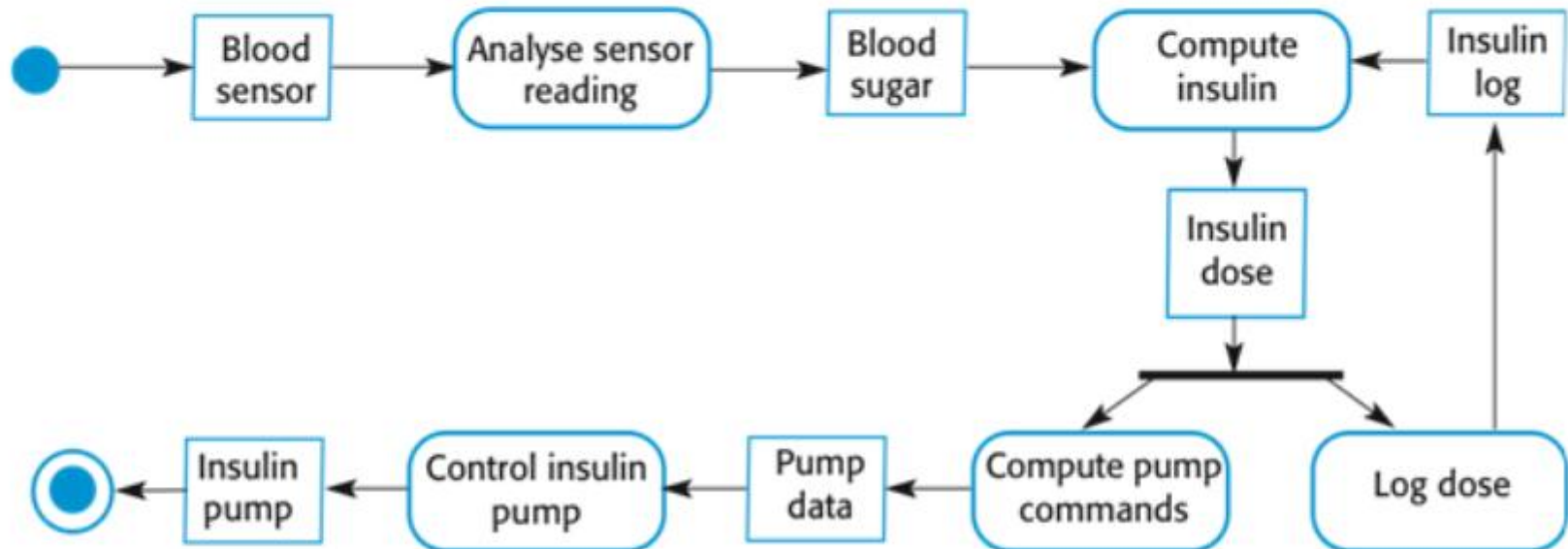
Insulin pump control system

- ✧ Collects data from a blood sugar sensor and calculates the amount of insulin required to be injected
- ✧ Calculation based on the rate of change of blood sugar levels
- ✧ Sends signals to a micro-pump to deliver the correct dose of insulin
- ✧ Safety-critical system as low blood sugars can lead to brain malfunctioning, coma and death; high-blood sugar levels have long-term consequences such as eye and kidney damage

Insulin pump hardware architecture



Activity model of the insulin pump



Essential high-level requirements

- ✧ The system shall be available to deliver insulin when required.
- ✧ The system shall perform reliably and deliver the correct amount of insulin to counteract the current level of blood sugar.
- ✧ The system must therefore be designed and implemented to ensure that the system always meets these requirements.

MENTCARE: A PATIENT INFORMATION SYSTEM FOR MENTAL HEALTH CARE



- ✧ A patient information system to support mental health care is a medical information system that maintains information about patients suffering from mental health problems and the treatments that they have received
- ✧ Most mental health patients do not require dedicated hospital treatment but need to attend specialist clinics regularly where they can meet a doctor who has detailed knowledge of their problems
- ✧ To make it easier for patients to attend, these clinics are not just run in hospitals. They may also be held in local medical practices or community centres.

MENTCARE



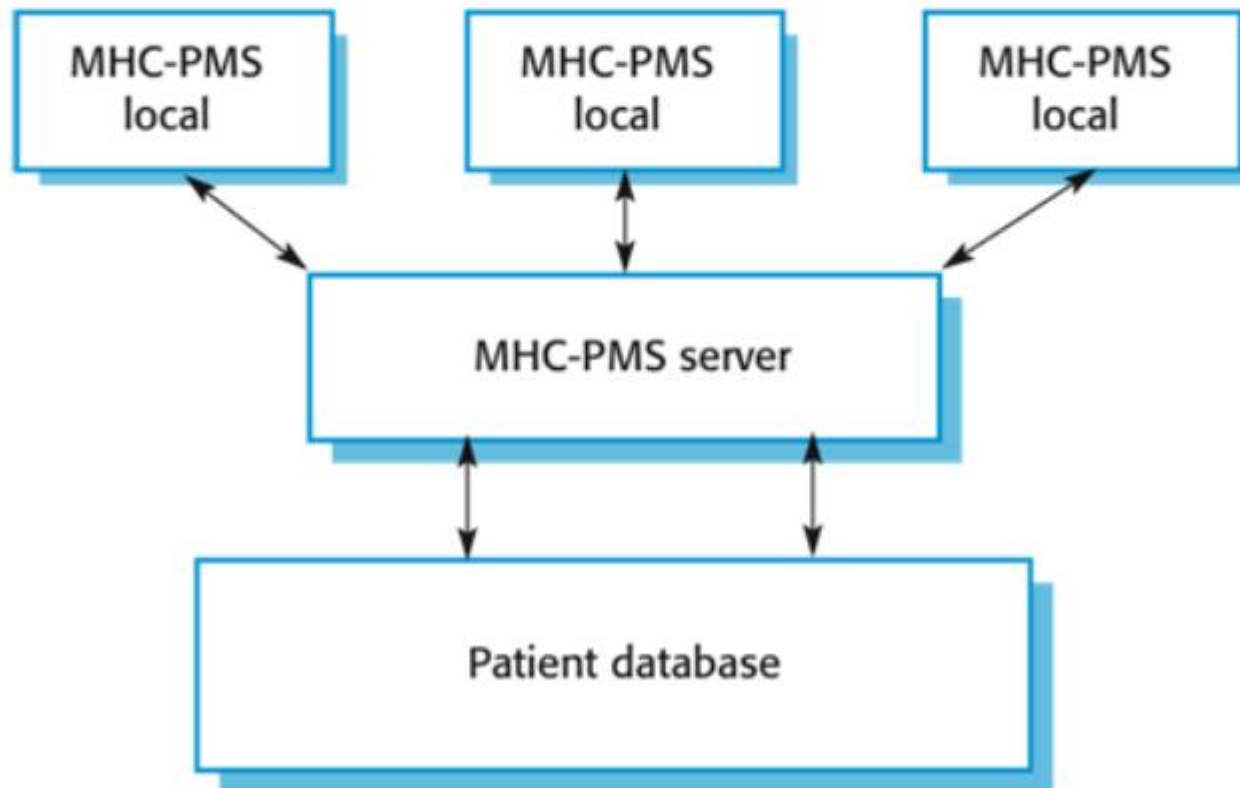
- ✧ The MHC-PMS (Mental Health Care-Patient Management System) is an information system that is intended for use in clinics
- ✧ It makes use of a centralized database of patient information but has also been designed to run on a PC, so that it may be accessed and used from sites that do not have secure network connectivity
- ✧ When the local systems have secure network access, they use patient information in the database but they can download and use local copies of patient records when they are disconnected

MHC-PMS goals



- ✧ To generate management information that allows health service managers to assess performance against local and government targets.
- ✧ To provide medical staff with timely information to support the treatment of patients.

The organization of the MHC-PMS



MHC-PMS key features

✧ Individual care management

- Clinicians can create records for patients, edit the information in the system, view patient history, etc. The system supports data summaries so that doctors can quickly learn about the key problems and treatments that have been prescribed.

✧ Patient monitoring

- The system monitors the records of patients that are involved in treatment and issues warnings if possible problems are detected

✧ Administrative reporting

- The system generates monthly management reports showing the number of patients treated at each clinic, the number of patients who have entered and left the care system, number of patients sectioned, the drugs prescribed and their costs, etc.

MHC-PMS concerns

✧ Privacy

- It is essential that patient information is confidential and is never disclosed to anyone apart from authorised medical staff and the patient themselves

✧ Safety

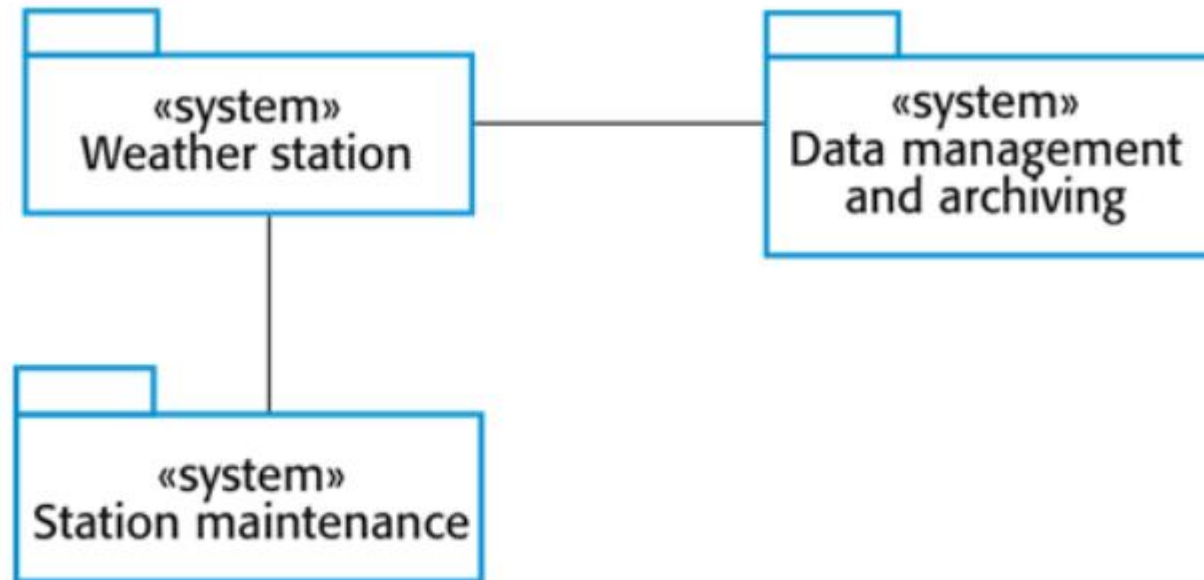
- Some mental illnesses cause patients to become suicidal or a danger to other people. Wherever possible, the system should warn medical staff about potentially suicidal or dangerous patients.
- The system must be available when needed otherwise safety may be compromised and it may be impossible to prescribe the correct medication to patients

Wilderness weather station



- ✧ The government of a country with large areas of wilderness decides to deploy several hundred weather stations in remote areas
- ✧ Weather stations collect data from a set of instruments that measure temperature and pressure, sunshine, rainfall, wind speed and wind direction
 - The weather station includes a number of instruments that measure weather parameters such as the wind speed and direction, the ground and air temperatures, the barometric pressure and the rainfall over a 24-hour period. Each of these instruments is controlled by a software system that takes parameter readings periodically and manages the data collected from the instruments.

The weather station's environment



Weather information system



✧ The weather station system

- This is responsible for collecting weather data, carrying out some initial data processing and transmitting it to the data management system.

✧ The data management and archiving system

- This system collects the data from all of the wilderness weather stations, carries out data processing and analysis and archives the data.

✧ The station maintenance system

- This system can communicate by satellite with all wilderness weather stations to monitor the health of these systems and provide reports of problems.

Additional software functionality



- ✧ Monitor the instruments, power and communication hardware and report faults to the management system.
- ✧ Manage the system power, ensuring that batteries are charged whenever the environmental conditions permit but also that generators are shut down in potentially damaging weather conditions, such as high wind.
- ✧ Support dynamic reconfiguration where parts of the software are replaced with new versions and where backup instruments are switched into the system in the event of system failure.

Key points



- ✧ Software engineers have **responsibilities** to the engineering profession and society. They should not simply be concerned with technical issues.
- ✧ Professional societies publish **codes of conduct** which set out the **standards of behaviour** expected of their members.
- ✧ Three case studies are used in the book:
 - An embedded insulin pump control system
 - A system for mental health care patient management
 - A wilderness weather station

Key points



- ✧ Software engineers have **responsibilities** to the engineering profession and society. They should not simply be concerned with technical issues.
- ✧ Professional societies publish **codes of conduct** which set out the **standards of behaviour** expected of their members.
- ✧ Three case studies are used in the book:
 - An embedded insulin pump control system
 - A system for mental health care patient management
 - A wilderness weather station

ILEARN: A DIGITAL LEARNING ENVIRONMENT

- A digital learning environment is a framework in which a set of general-purpose and specially designed tools for learning may be embedded plus a set of applications that are geared to the needs of the learners using the system.
- The tools included in each version of the environment are chosen by teachers and learners to suit their specific needs.
 - These can be general applications such as spreadsheets, learning management applications such as a Virtual Learning Environment (VLE) to manage homework submission and assessment, games and simulations.

SERVICE-ORIENTED SYSTEMS

- The system is a service-oriented system with all system components considered to be a replaceable service.
- This allows the system to be updated incrementally as new services become available.
- It also makes it possible to rapidly configure the system to create versions of the environment for different groups such as very young children who cannot read, senior students, etc.

ILEARN SERVICES

- *Utility services* that provide basic application-independent functionality and which may be used by other services in the system.
- *Application services* that provide specific applications such as email, conferencing, photo sharing etc. and access to specific educational content such as scientific films or historical resources.
- *Configuration services* that are used to adapt the environment with a specific set of application services and do define how services are shared between students, teachers and their parents.

ILEARN ARCHITECTURE

Browser-based user interface

iLearn app

Configuration services

Group
management

Application
management

Identity
management

Application services

Email Messaging Video conferencing Newspaper archive
Word processing Simulation Video storage Resource finder
Spreadsheet Virtual learning environment History archive

Utility services

Authentication
User storage

Logging and monitoring
Application storage

Interfacing
Search

ILEARN SERVICE INTEGRATION

- *Integrated services* are services which offer an API (application programming interface) and which can be accessed by other services through that API. Direct service-to-service communication is therefore possible.
- *Independent services* are services which are simply accessed through a browser interface and which operate independently of other services. Information can only be shared with other services through explicit user actions such as copy and paste; re-authentication may be required for each independent service.

Key points



- ✧ Software engineers have **responsibilities** to the engineering profession and society. They should not simply be concerned with technical issues.
- ✧ Professional societies publish **codes of conduct** which set out the **standards of behaviour** expected of their members.
- ✧ Three case studies are used in the book:
 - An embedded insulin pump control system
 - A system for mental health care patient management
 - A wilderness weather station

Thank you